

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART XIV

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

Part - I:	Overview of Agentic Coding Assistants
Part – II:	Claude Ecosystem, Installing & using Claude Code
Part – III:	Slash Commands, Context Window & `.claude` Directory
Part – IV:	Workflow of Building a Basic Vulnerable Web Application
Part – V:	Exploiting Vulnerable Web Application
Part – VI:	Workflow of Implementing Features and Fixing Bugs
Part – VII:	Collaborative Workflow: Claude Code + GitHub + Jira
Part – VIII:	Built-in Skills & Vuln-Web-App Current Status
Part – IX:	Creating & using Custom Skills & Skill Marketplace
Part – X:	Deep Dive: Conceptual Understanding of MCP
Part – XI:	MCP in Action
Part – XII:	Creating / Using Built-in & Custom Subagents
Part – XIII:	Git Worktrees, Agent Teams & a revisit to OWASP Top 10
Part – XIV:	**A2A Protocol with a Hands-on Demonstration**

Important Concepts

A2A Protocol

Instructor: Muhammad Arif Butt, PhD

MCP vs A2A: Two Protocols one Agentic Stack

MODEL CONTEXT PROTOCOL

MCP

Agent → Tools, Data & APIs

Launched by **Anthropic in November 2024** as an open standard for connecting AI assistants directly to the systems where data lives — files, databases, and business tools — eliminating one-off custom integrations.

WHAT IT STANDARDIZES

MCP standardizes how an agent connects to its tools, resources and APIs to retrieve information and take action.

e.g. An agent uses MCP to call a weather API tool and fetch a live forecast.



https://modelcontextprotocol.io

AGENT2AGENT PROTOCOL

A2A

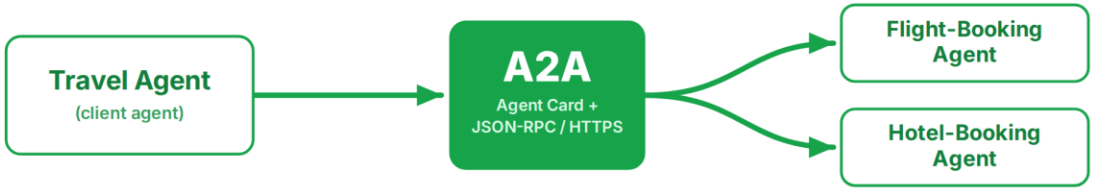
Agent → Peer Agent

Launched by **Google in April 2025,** then donated to the **Linux Foundation in June 2025,** making it open-source and community-governed. **IBM's Agent Communication Protocol (ACP)** merged into A2A in **August 2025,** unifying the agent-to-agent standard.

WHAT IT STANDARDIZES

A2A standardizes how an agent connects to peer agents to delegate tasks and collaborate on work.

e.g. A travel-planning agent asks a flight-booking agent to find available flights.

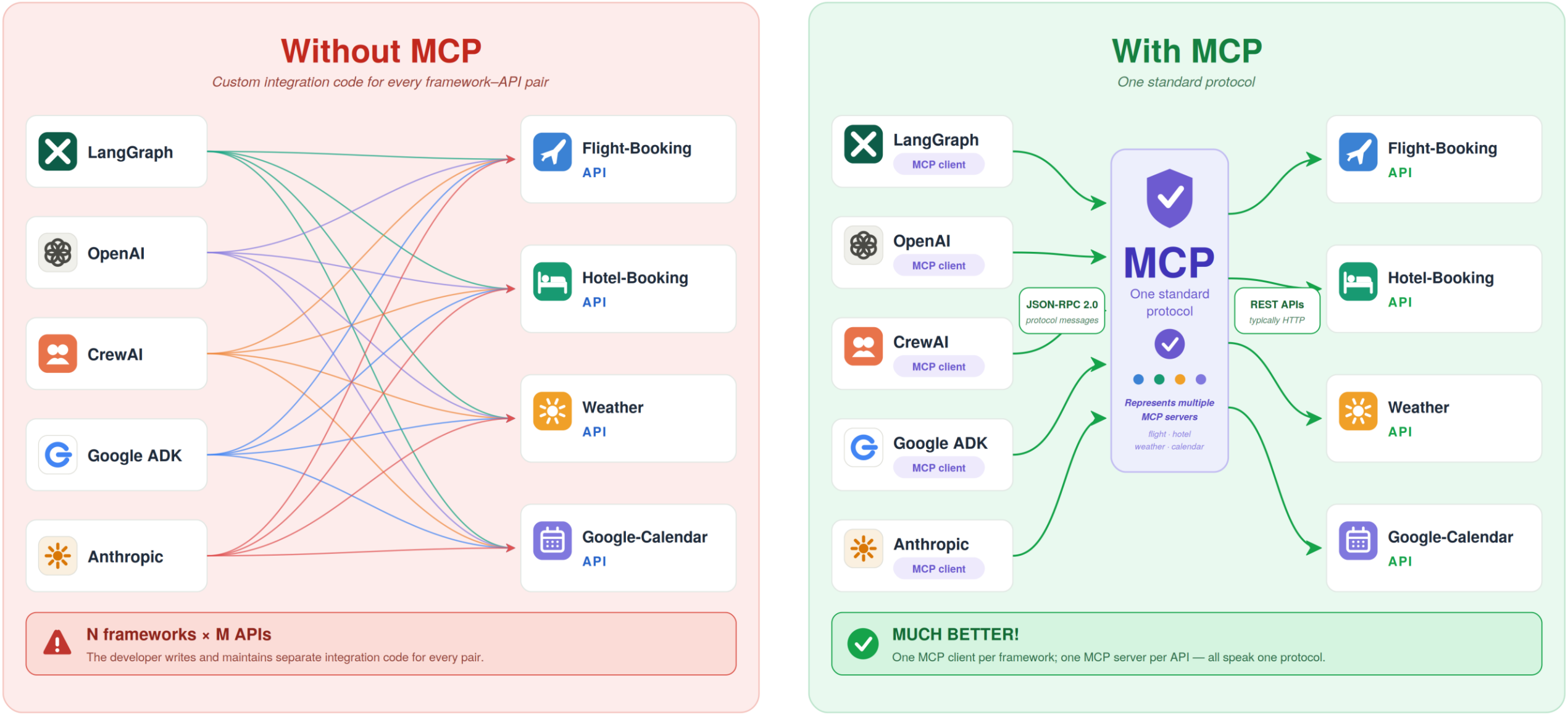


https://a2a-protocol.org/latest/

Complementary, not competing.

MCP connects an agent to its tools and data; **A2A** connects agents to one another — together they form the connective tissue of agentic AI.

Why MCP? (A Recap)

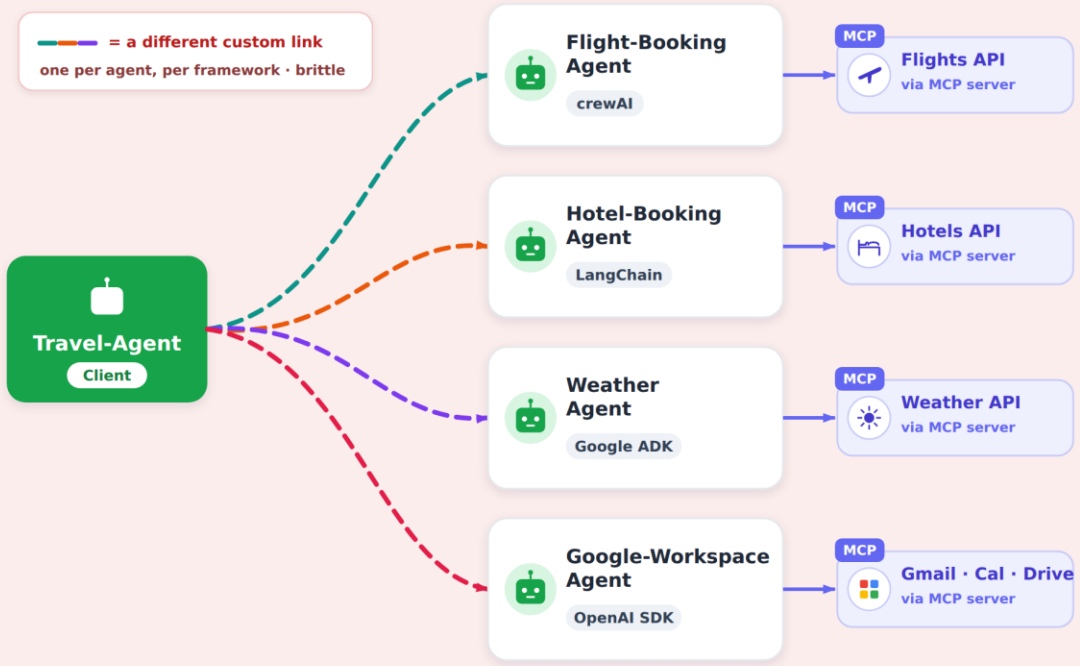


Why A2A?

Same four agents, same trip to book — only the wiring between them changes (their MCP tools stay the same).

Without A2A

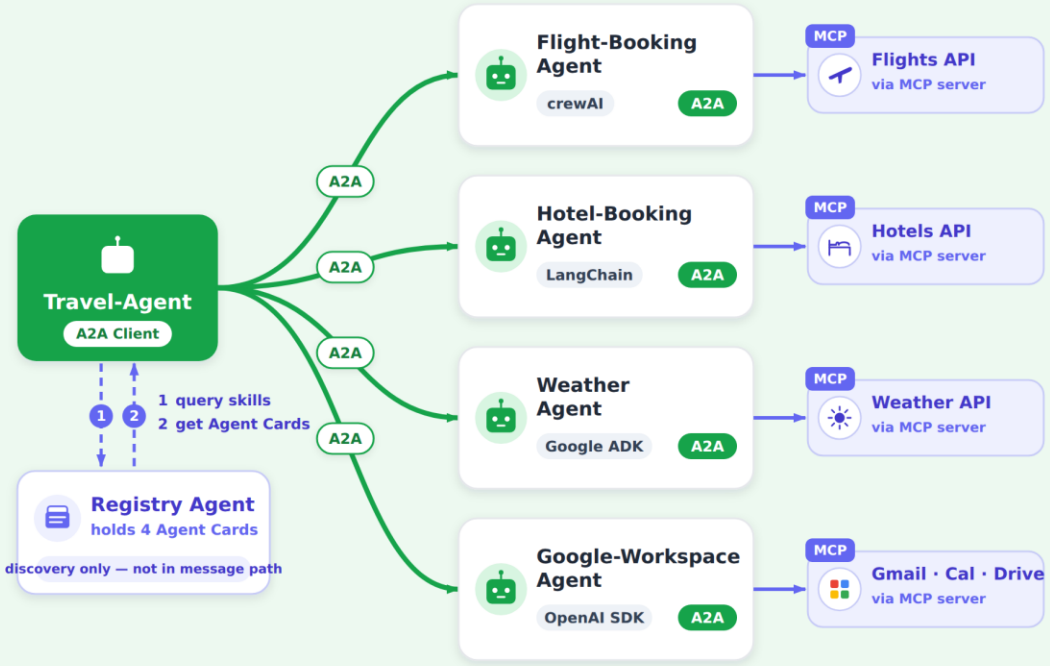
Every agent-to-agent link is bespoke, framework-specific glue.



4 agents = 4 bespoke integrations.
Each hand-built for a different framework. Add an agent → write another. Brittle.

With A2A

Discover agents via the registry, then message them directly.



Speak A2A once → reach every agent.
Add an agent = one integration, any framework. The registry handles discovery.

The MCP tool layer is identical on both sides.
Each agent reaches its own tools through its own MCP server — A2A only standardizes the agent-to-agent wiring, never the agent-to-tools wiring.

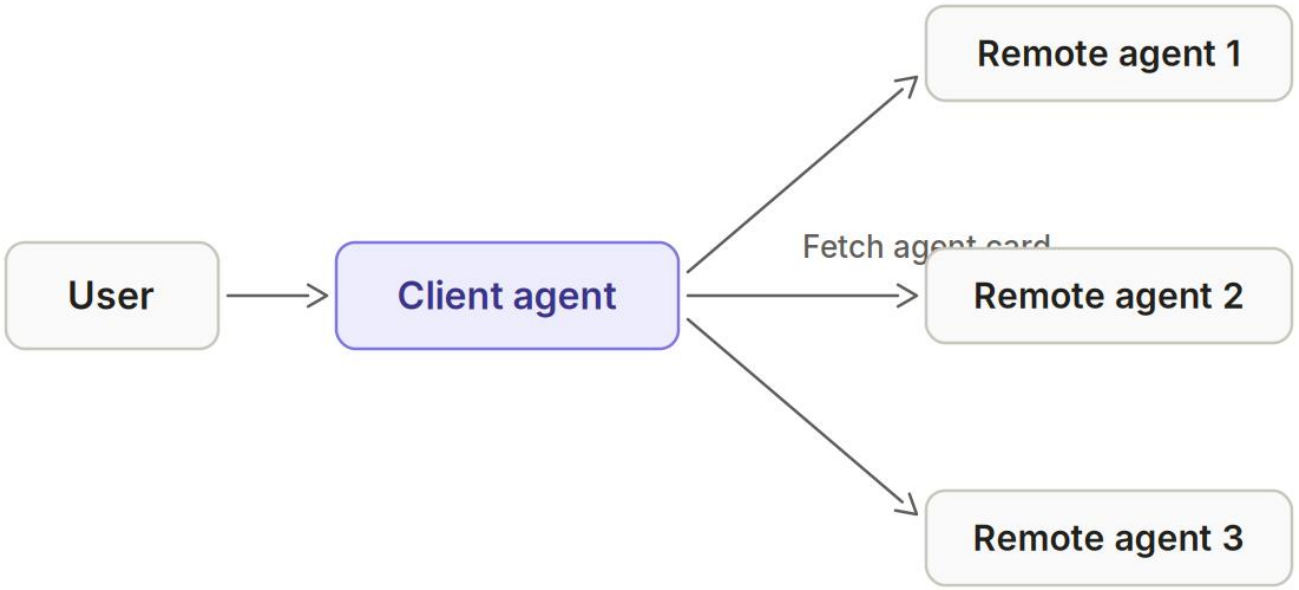
A2A Client Server Communication

STEP 1

Agent card lookup

Discovery

Client agent looks up remote agents to find the best fit for the task



1. A user or another AI agent sends a request to the client agent
2. The client agent looks up candidate remote agents for the job
3. It fetches each one's Agent Card — a JSON profile of skills, endpoint, and auth needs

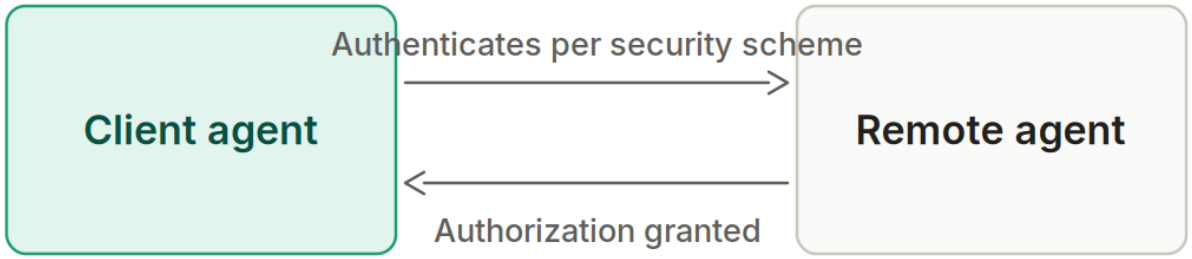
A2A Client Server Communication

STEP 2

OAuth 2.0 / API keys / OIDC

Authentication

Client agent authenticates per the security scheme in the agent card



1. The client agent authenticates using the security scheme named in the Agent Card
2. Supported schemes include API keys, OAuth 2.0, and OpenID Connect
3. Once verified, the remote agent grants authorization and access

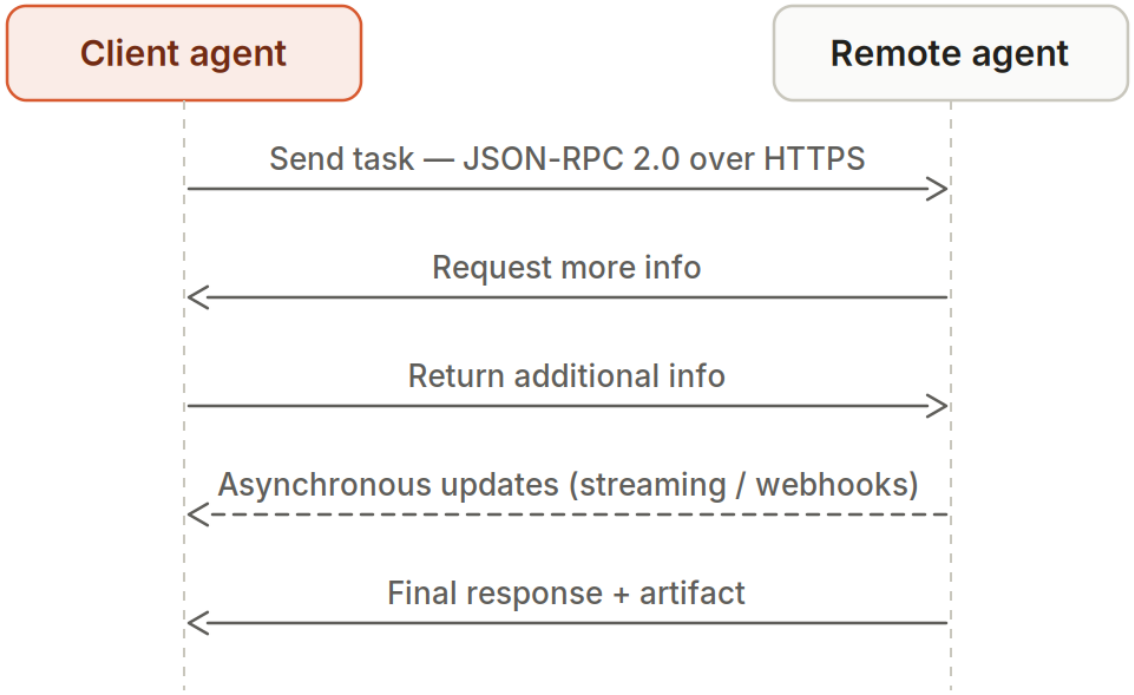
A2A Client Server Communication

STEP 3

JSON-RPC 2.0 over HTTPS

Communication

Client sends a task; remote agent processes it and returns results



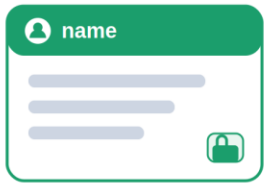
1. Client sends a task via JSON-RPC 2.0 over HTTPS; the remote agent may request more info
2. Long-running tasks get asynchronous updates via streaming or webhooks
3. On completion, the remote agent returns a final response and artifact

Four Building Blocks of A2A

A2A in one picture

The four building blocks — how two agents discover each other, agree on work, talk, and deliver a result.

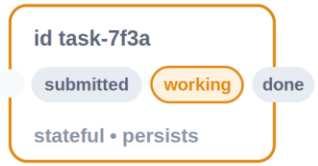
1 DISCOVER — Agent Card



A server agent publishes a JSON card so any client can read its skills, endpoint and auth.

name · skills · url · auth

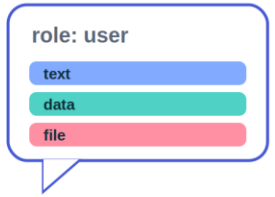
2 DELEGATE — Task



A shared, stateful ticket the client opens — it carries an id and moves through its states.

id · state · from/to · payload

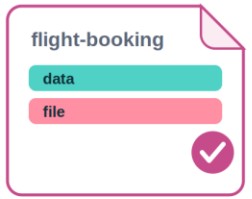
3 EXCHANGE — Messages / Parts



Agents talk in turns. Each message is a role plus a list of typed Parts.

text · data · file

4 DELIVER — Artifact



The finished output, returned as a named bundle of the same typed Parts.

data · file · result

How it all fits together — booking a flight Lahore → Karachi

1 · **Agent Card** — Travel Agent reads the Flight Agent's card

2 · **Task** — opens a task to book Lahore → Karachi

3 · **Messages** — "Which cabin?" → "economy"

4 · **Artifact** — receives the e-ticket back

Discover → delegate → exchange → deliver.

Agent Card + Task + Messages + Artifact are all any two agents need to collaborate — across any framework or vendor.

What is Agent Card?

Agent Card

A digital business card an agent publishes as JSON at a well-known URL — so any other agent can discover it.



What the card tells you

1. **Identity**
   name, description, provider — who the agent is
2. **Endpoint**
   url + version — where to reach it & which spec
3. **Capabilities**
   streaming, push, input/output modes it supports
4. **Security**
   securitySchemes — how to authenticate with it
5. **Skills**
   id, name, tags, examples — what it can actually do

flights.example.com /.well-known/agent-card.json

```
{
  "name": "Flight Booking Agent",
  "description": "Searches & books flights on 400+ airlines.",
  "provider": { "organization": "SkyBook Inc." },
  "url": "https://flights.example.com/a2a",
  "version": "1.2.0",
  "capabilities": { "streaming": true, "pushNotifications": true },
  "defaultInputModes":  ["text", "data"],
  "defaultOutputModes": ["text", "data"],
  "securitySchemes": {
    "bearer": { "type": "http", "scheme": "bearer" }
  },
  "skills": [
    {
      "id": "search-flights",
      "name": "Search Flights",
      "description": "Find flights by route & date.",
      "tags": ["travel", "flights"],
      "examples": ["Flights Lahore -> Karachi"]
    }
  ]
}
```

Read the card, know the agent.
One standard JSON file lets any client agent discover a server agent's identity, skills, and how to call it — no custom integration.

What is Task?

Task

The unit of work — a shared ticket both agents can see and update in real time. Stateful, and it persists across exchanges.

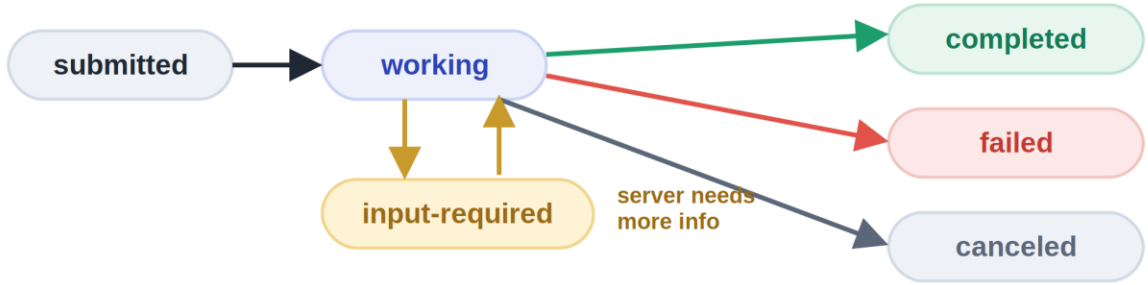
task.json

```
{
  "id": "task-7f3a9c21",
  "kind": "task",
  "from": "travel-agent",
  "to": "flight-booking-agent",
  "status": {
    "state": "input-required",
    "createdAt": "2026-07-16T10:32:04Z",
    "updatedAt": "2026-07-16T10:32:19Z"
  },
  "payload": {
    "route": "Lahore -> Karachi",
    "date": "2026-07-24",
    "passengers": 1
  }
}
```

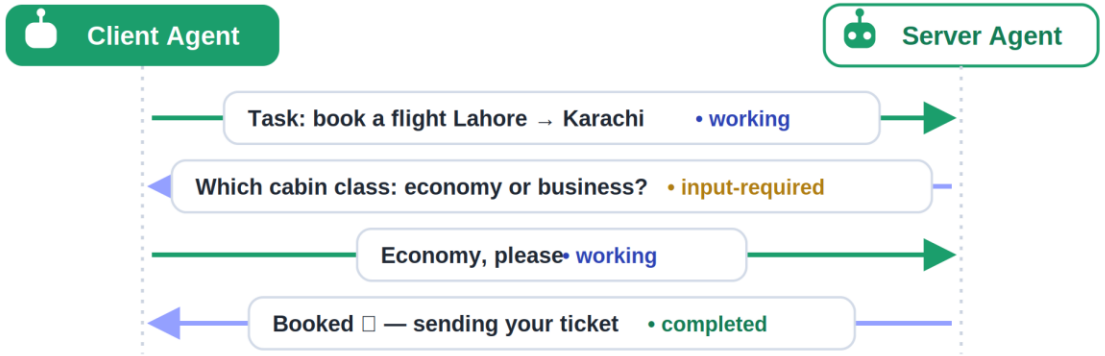
id – unique ticket • kind – object type • from / to – sender ↔ recipient

state – lifecycle position • createdAt / updatedAt – timestamps • payload – the work

Task lifecycle — the state field



One task, many exchanges



A living ticket, not a one-shot call.

A Task carries a unique id and a state that moves submitted → working → completed — pausing at input-required whenever the server needs more.

What is Messages / Parts?

Messages / Parts

A message is one turn between agents. Its content is a list of Parts — each Part a typed piece: text, data, or a file.



A message is a list of typed Parts

1. **TextPart**
   kind: "text" — natural-language text
2. **DataPart**
   kind: "data" — structured JSON: forms, options, parameters
3. **FilePart**
   kind: "file" — a file by URI or bytes (PDF, image, audio)
4. **role & mixing**
   role = who speaks (user / agent); one message can mix many Parts

message.json

```
{
  "role": "user",
  "taskId": "task-7f3a9c21",
  "parts": [
    {
      "kind": "text",
      "text": "Book a flight Lahore -> Karachi on 24 Jul."
    },
    {
      "kind": "data",
      "data": { "route": "Lahore -> Karachi", "cabin": "economy" }
    },
    {
      "kind": "file",
      "file": {
        "name": "passport.pdf",
        "mimeType": "application/pdf"
      }
    }
  ]
}
```

Messages carry the conversation.
Every message is a role plus a list of typed Parts — text, data, or files — so agents can exchange anything in a single turn.

What is Artifact?

Artifact

The finished output a server agent produces for a task — a named bundle, also built from typed Parts.



What an Artifact is

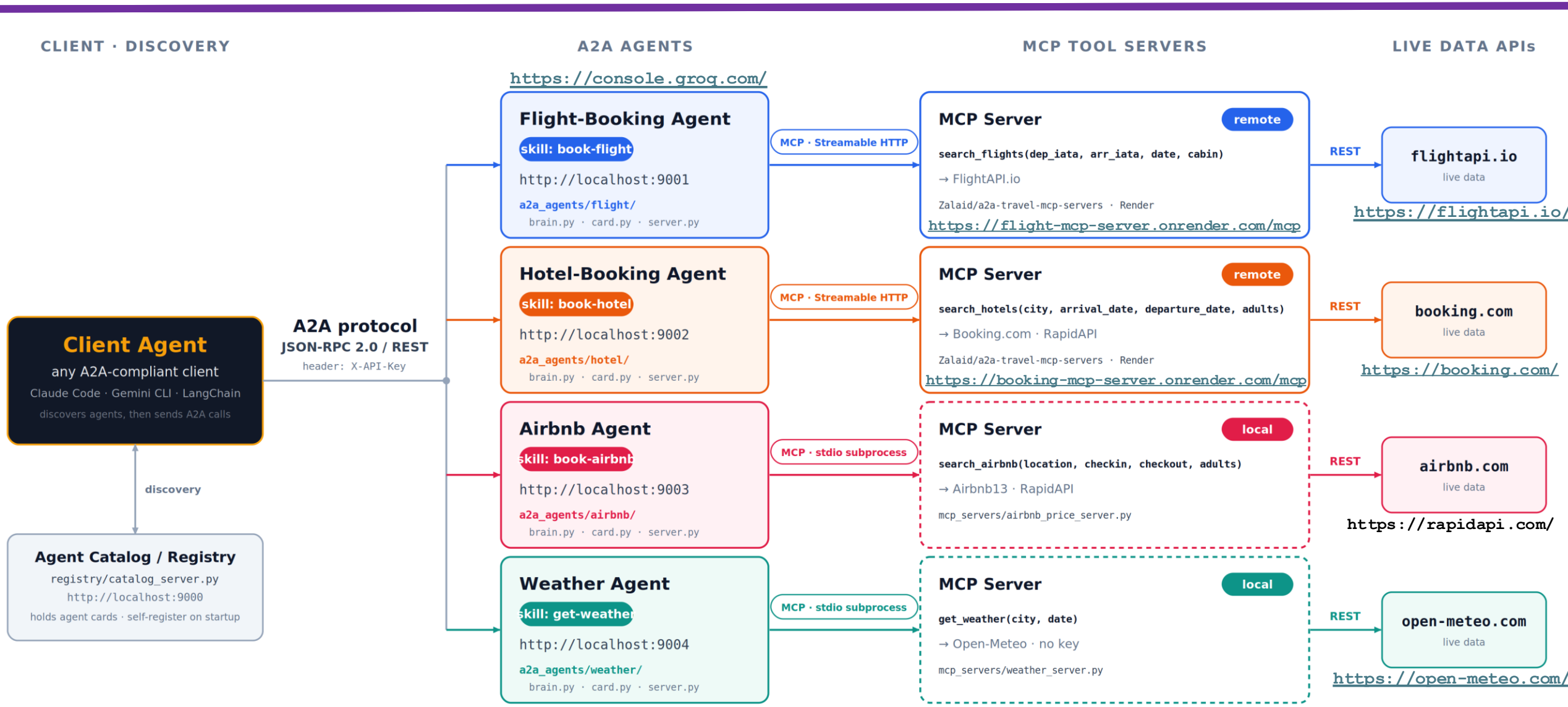
1. **The task's result**
   the deliverable a server produces once the work is done
2. **Built from Parts**
   same typed Parts as messages — data, files, or text
3. **Streams in chunks**
   large artifacts can arrive incrementally, then finalize
4. **Message vs Artifact**
   messages carry the conversation; artifacts carry the output

artifact.json

```
{
  "artifactId": "art-5c1a92e0",
  "name": "flight-booking",
  "description": "Confirmed itinerary Lahore -> Karachi",
  "parts": [
    {
      "kind": "data",
      "data": { "pnr": "XK7Q2P", "flight": "PK-303",
                "seat": "14C", "price": "PKR 18,500" }
    },
    {
      "kind": "file",
      "file": {
        "name": "eticket.pdf",
        "mimeType": "application/pdf",
        "uri": "https://sky.example/eticket.pdf"
      }
    }
  ]
}
```

Artifacts are the deliverables.
When a task finishes, the server returns its result as an Artifact — a named bundle of Parts the client can use directly.

Travel Agent App: A2A & MCP

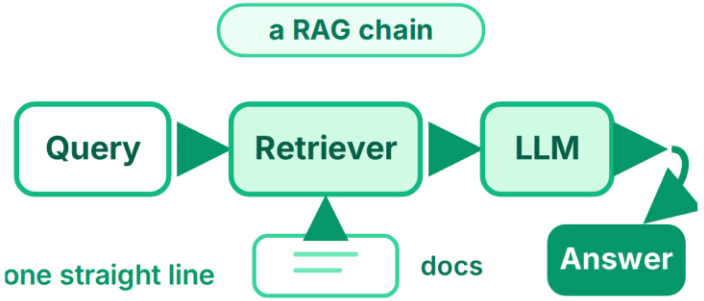


Hands on Practice: A2A



The LangChain Ecosystem

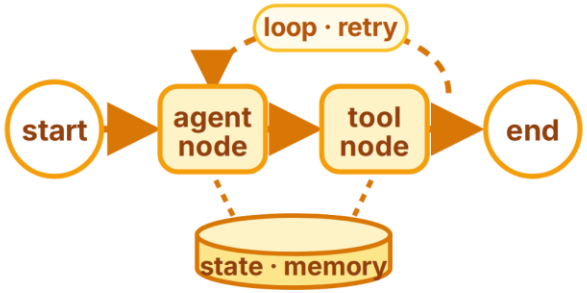
1 BUILD LangChain



A framework to build **simple linear workflows** called **chains** — e.g. a **RAG-based** AI application.

chains · linear · RAG · LLM

2 ORCHESTRATE LangGraph



A framework to **orchestrate multi-step, stateful workflows** using LLMs with a **graph-based** structure — supporting **loops, retries & memory**.

stateful · loops · retries · memory

3 OBSERVE LangSmith



A **tracing, debugging & observability** platform for agentic LLM apps.

- **Tracing** — track a prompt's complete journey inside your app
- **Debugging** — find why the model gave an incorrect response
- **Observability** — monitor performance in real time

traces · debug · monitor

Wire up LangSmith — 5-step setup

1. **1 · Sign up** — create an account at **smith.langchain.com**
2. **2 · Create API key** — generate it in the LangSmith settings
3. **3 · Add key to .env** — paste it in `.env` in your project root
4. **4 · Enable tracing** — set `LANGSMITH_TRACING=True`
5. **5 · Run & view** — run your app — traces appear in the web UI

Q/A



Thank you!